

# **Text Analytics Application in Business**

# Agenda

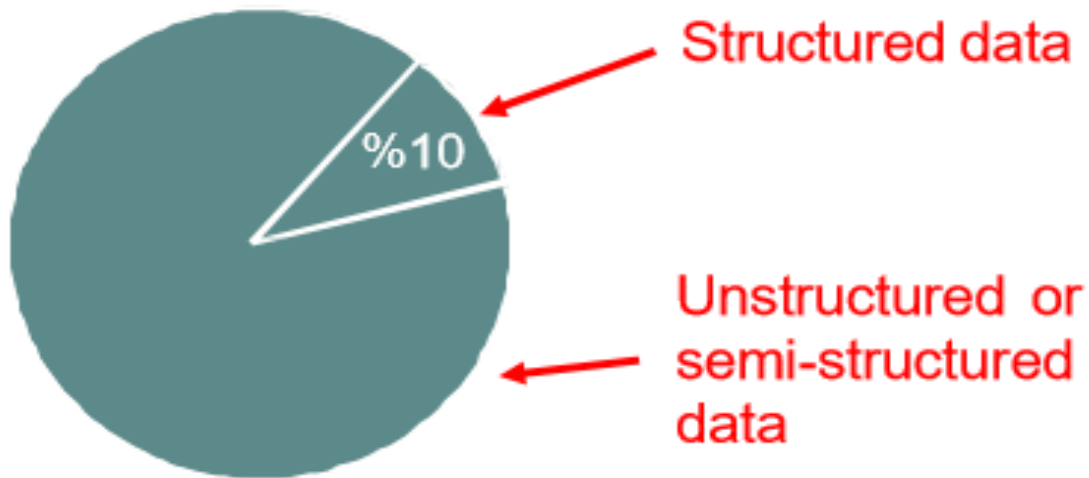
---

- What is Text Mining?
- Application of text mining in business.
- Key Challenges with Text Analysis
- Key Tasks in Text Analysis
- Text Mining process
- Sentiment Analysis

# Motivation for Text Mining

**Approximately 90% of the world's data is held in unstructured formats.**

Source: Oracle Corporation

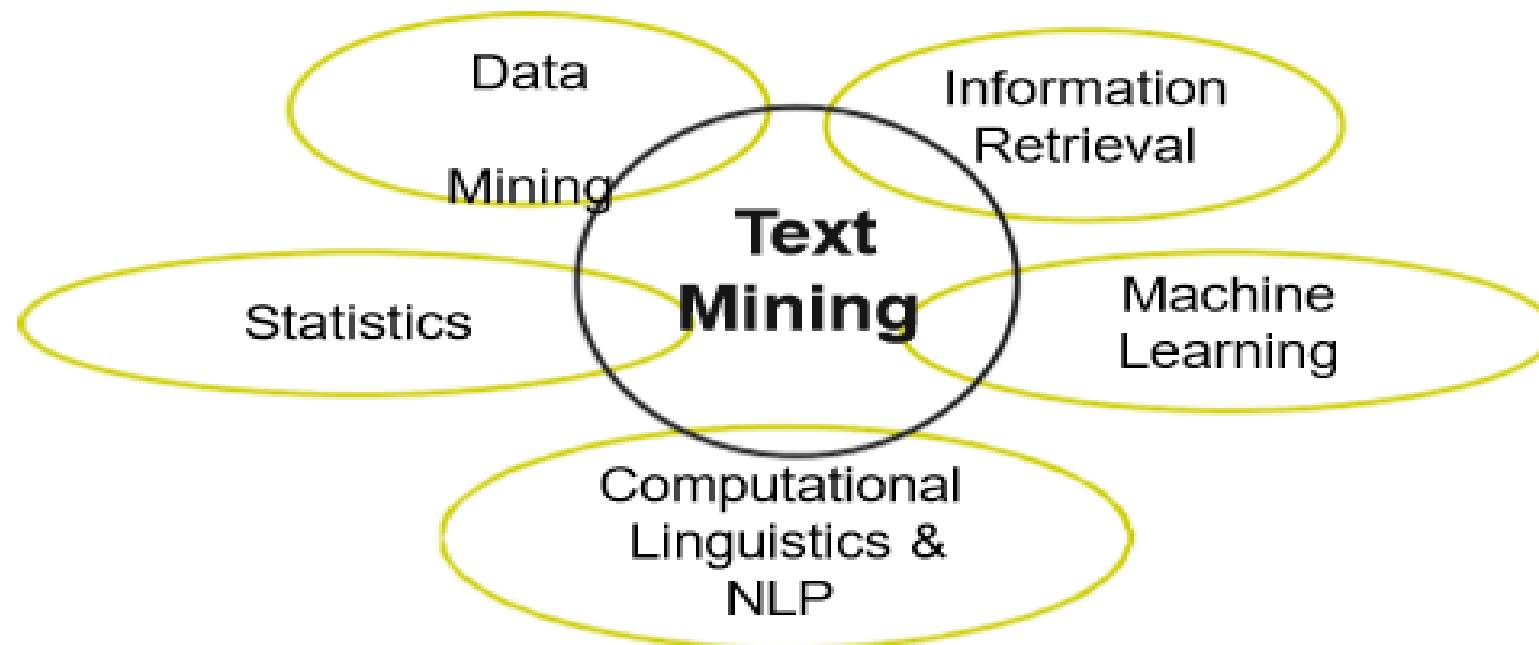


Examples:

- web pages
- emails
- customer complaint letters
- corporate documents
- scientific papers
- books in digital libraries

# Text Mining

The extraction of implicit, **previously unknown** and **potentially useful** information from large amounts of **textual resources**.



# Text Mining



- Text mining is the application of data-mining techniques to text files. Text mining includes sentiment analysis, text clustering, and text classification. Companies use text data for a wide range of applications, including:
  - National security applications
  - Document or email classification
  - Customer sentiment analysis
  - Natural-language processing
- Text analysis is essentially the processing and representation of data that is in text form to analyze it and learn new models from it.

# Application of text mining in business

---

## Sentiment Analysis:

- **Objective:** Analyzing text to determine the sentiment (positive, negative, neutral) expressed by customers or the public.
- **Application:** Helps businesses gauge customer satisfaction, identify issues, and respond to public opinion. Useful in social media monitoring, customer reviews, and feedback analysis.

# Application of text mining in business

---

## Customer Feedback Analysis:

- **Objective:** Analyzing customer reviews, surveys, and feedback.
- **Application:** Extracts insights from customer comments to identify trends, common issues, and areas for improvement in products or services.

## Social Media Monitoring:

- **Objective:** Monitoring social media channels for brand mentions, trends, and customer sentiment.
- **Application:** Helps businesses understand how their brand is perceived, identify influencers, and respond to customer concerns in real-time.

# Application of text mining in business

---

## Competitor Analysis:

- **Objective:** Analyzing online content related to competitors.
- **Application:** Helps businesses track competitor mentions, compare product reviews, and identify market trends.

## Employee Feedback and Engagement:

- **Objective:** Analyzing employee surveys, reviews, and feedback.
- **Application:** Provides insights into employee satisfaction, identifies areas for improvement in the workplace, and helps in employee engagement initiatives



# Application of text mining in business

---

## Survey and Feedback Analysis:

- **Objective:** Analyzing open-ended responses in surveys.
- **Application:** Helps businesses gain deeper insights from survey data by categorizing and analyzing qualitative responses.

## Trend Analysis:

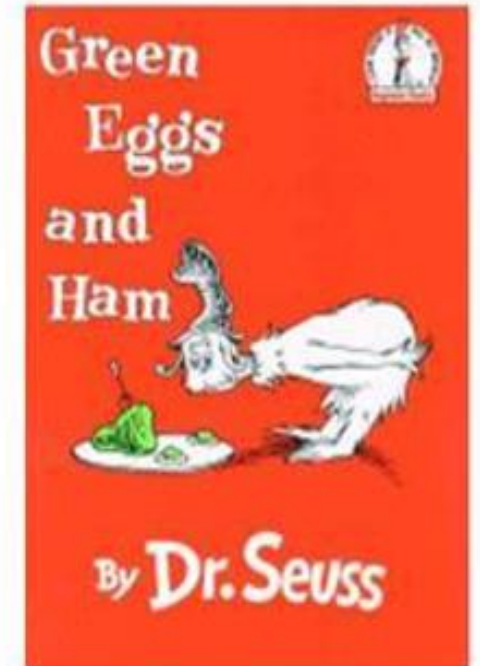
- **Objective:** Identifying emerging trends or topics from textual data.
- **Application:** Enables businesses to stay informed about industry trends, customer preferences, and emerging issues.

# Text Analysis

---

Encompasses the processing and representation of text for analysis and learning tasks

- Analyzing tweets
  - Twitter currently generates 500 MM tweets per day
  - It generates tweets in close to 40 languages
  - Distinct number of words can be close to 100,000
- High-dimensionality
  - Every distinct term is a dimension
  - Green Eggs and Ham: A 50-D problem!
- Data is unstructured



# Key Challenges with Text Analysis

---

- The main challenge in text analysis is the problem of ***high dimensionality***.
  - When you analyze a document, you find that every possible word in the document represents a dimension.
  - Processing data in such high dimensions requires a lot of computing power and memory. Algorithms can become very slow and inefficient.
- The other major challenge with text analysis is that the ***data is unstructured***. We first need to do a lot of pre-processing to transform it into a more structured format

# Text mining Process

## 1. Text Preprocessing

- syntactic and/or semantic analysis

## 2. Feature Generation

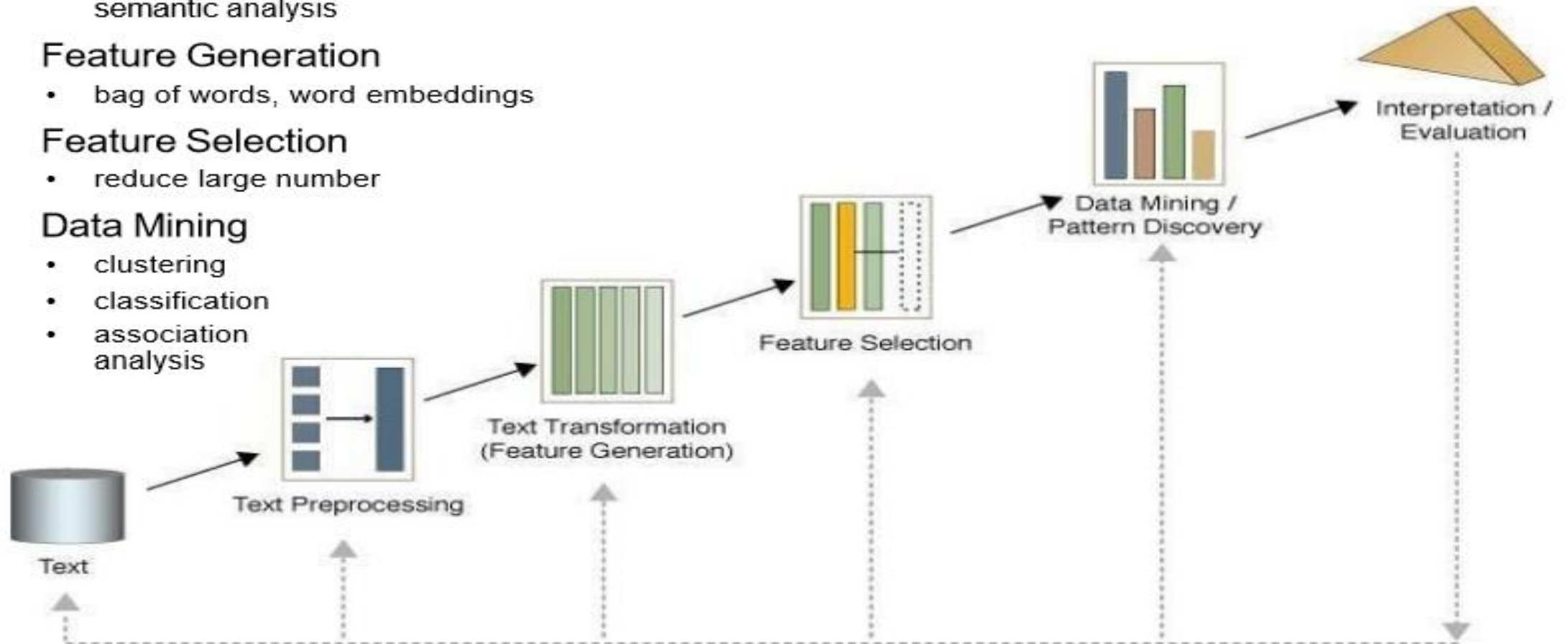
- bag of words, word embeddings

## 3. Feature Selection

- reduce large number

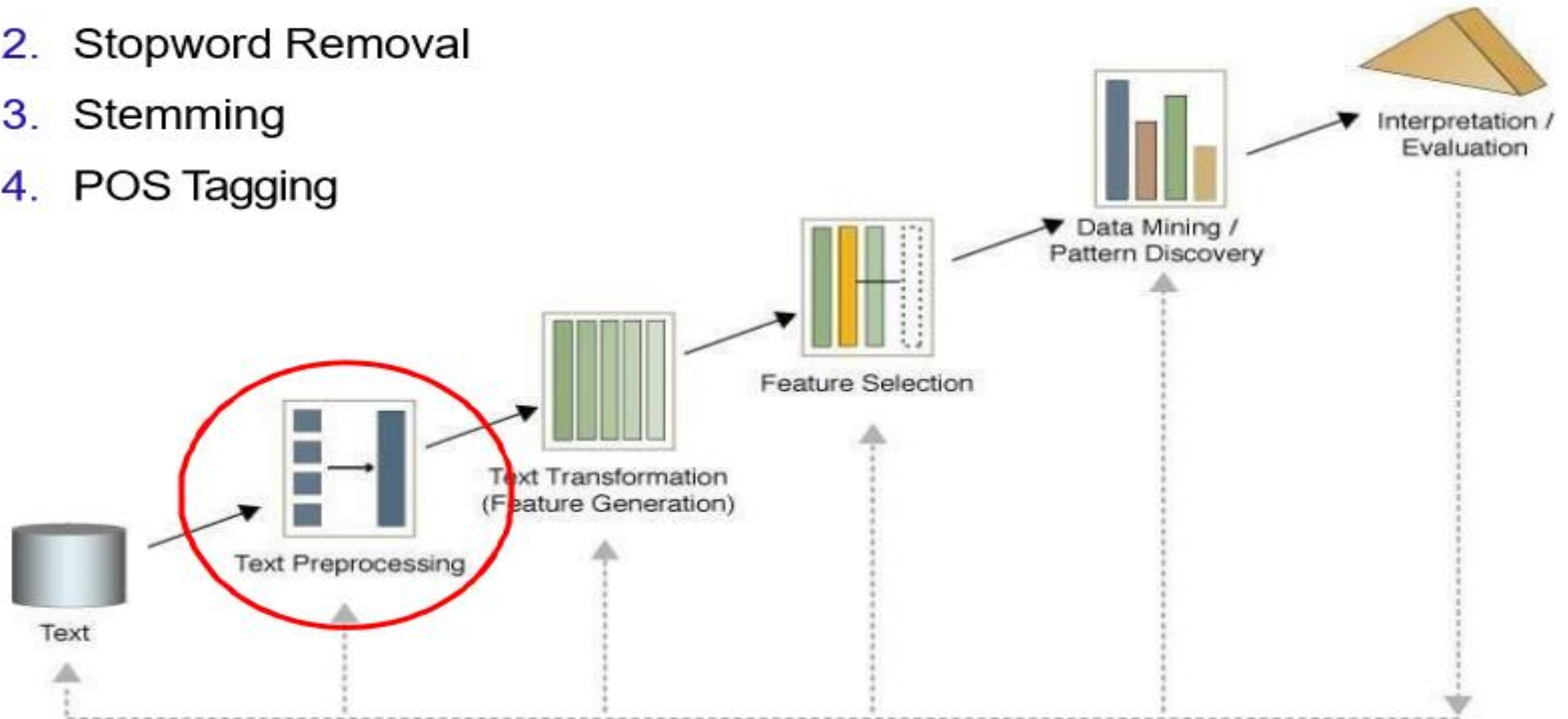
## 4. Data Mining

- clustering
- classification
- association analysis



# Text Preprocessing

1. Tokenization
2. Stopword Removal
3. Stemming
4. POS Tagging



# Text Preprocessing: Parsing and Tokenizing

---

- **Tokenizing** is the process of converting a sentence into a group of words.
- Any kind of separator can be used to find all the words in the sentence.
- If **space** is used as the **separator** of the tokens for "Hello world". you'd get ["Hello", "world"].
- Also, you can tokenize on punctuations marks (like commas, question marks, etc.) as points to split the sentence.

# Text Preprocessing: Parsing and Tokenizing

---

- Tokenizing based on punctuation marks might **not be well suited** to certain scenarios. because punctuation is often part of a word or has meaning within a word.
  - For example, if the text contains contractions such as "we'll", tokenizing based on punctuation splits them into separated words "we" and "ll")
- Tokenization is a much more difficult task than one may expect.
  - For example, should words such as state-of-the-art, Wi-Fi, and San Francisco be considered one token or more?
  - There isn't always a one-size-fits-all solution, and the best approach often depends on the specific task you're trying to achieve.

# Text Preprocessing:

## Case Folding

---

- Another text normalization technique is called ***case folding***.
- ***Case Folding*** reduces all letters to lowercase—or the opposite, if applicable. The goal is to make the computer treat "Apple" and "apple" as the same word.
- If implemented incorrectly, case folding may reduce or change the meaning of the text and create more noise.
  - for example, applying case folding to the name of a specific company "General Motors" becomes "general" and "motors", losing the crucial information that it's referring to a specific company.



# Text Preprocessing: Stop Word Removal

---

- Many of the most frequently used words in English are likely to be **useless** for text mining
- These words are called *Stopwords*
  - examples: the, of, and, to, an, is, that... ,
  - typically text contains about 400 to 500 such words
  - for an application, an additional domain specific stopwords list may be constructed
- Why should we remove stopwords?
  - Reduce data set size
    - stopwords account for 20-30% of total word count
  - Improve effectivity of text mining methods
    - stopwords may confuse the mining algorithm

# Text Preprocessing: Stemming

---

- Techniques to find the **stem of a word**
  - words: User, users, used, using → Stem: use
  - words: Engineering, engineered → Stem: engineer
- Usefulness for Text Mining
  - improve effectivity of text mining methods
    - matching of similar words
  - reduce term vector size
    - combining words with same stem may reduce the term vector as much as 40-50%

# Basic Stemming Rules

---

## – remove endings

- if a word ends with a consonant other than s, followed by an s, then delete **s**.
- if a word ends in es, drop the s.
- if a word ends in ing, delete the ing unless the remaining word consists only of one letter or of th
- if a word ends with **ed**, preceded by a consonant, delete the ed unless this leaves only a single letter
- .....

## – transform words

- if a word ends with “ies” but not “eies” or “aies” then “ies → y”

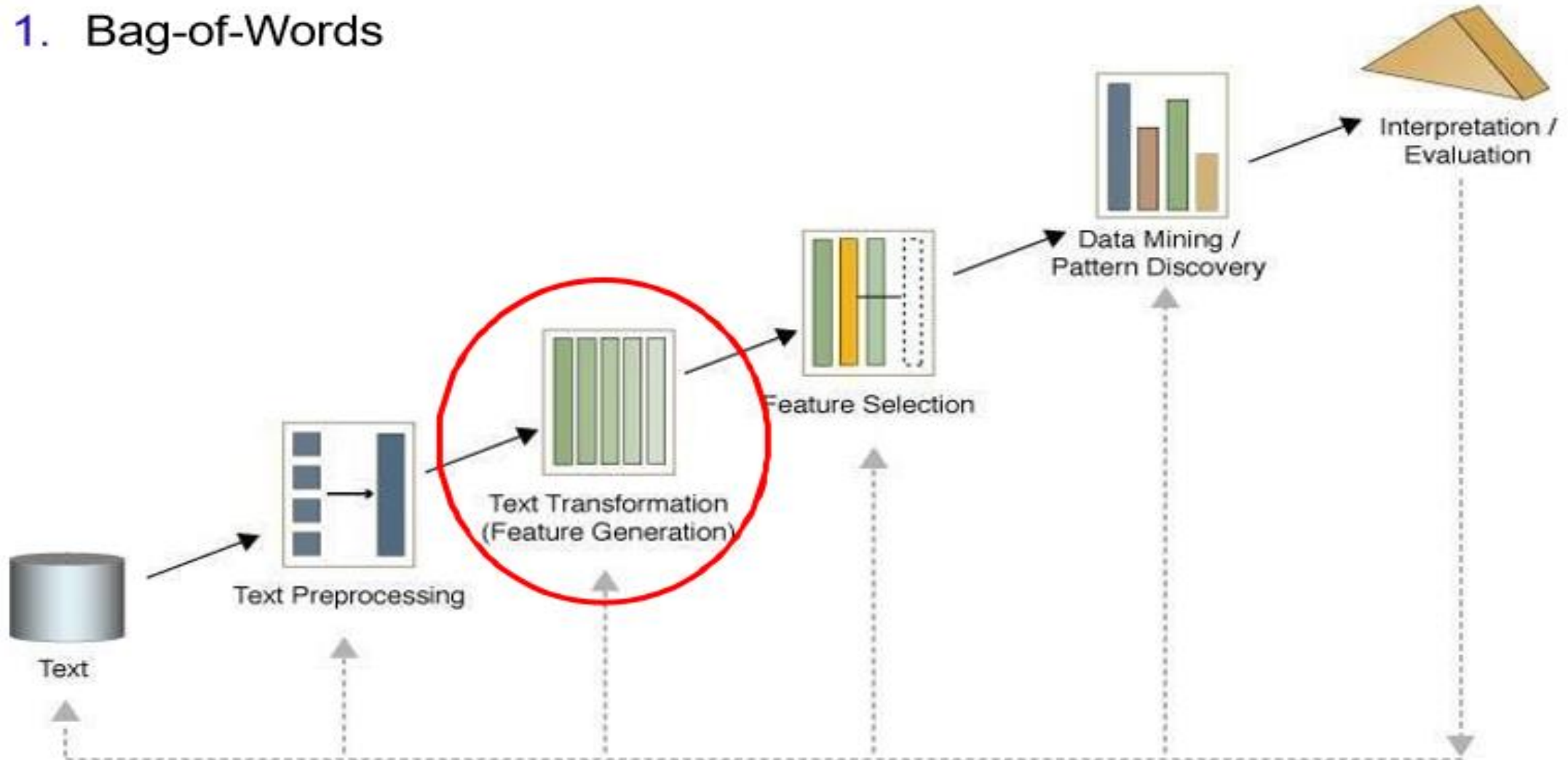
# Syntactic and Linguistic Text Preprocessing

---

- Simple Syntactic Processing
  - Text Cleanup (remove punctuation and HTML tags)
  - Tokenization (break text into single words or N-grams)
- Advanced Linguistic Processing
  - Word Sense Disambiguation
    - normalize synonyms (United States, USA, US)
  - Part Of Speech (POS) Tagging
    - parse sentences according to grammar
    - determine function of each term
    - e.g. John (noun) gave (verb) the (det) ball (noun)

# Feature Generation

## 1. Bag-of-Words



# Feature Generation

---

- Document is treated as a **bag of words** (or terms)
  - each word/term becomes a feature
  - order of words/terms is ignored
- Each document is represented by a vector
- Different techniques for vector creation:
  1. **Binary Term Occurrence**: Boolean attributes describe whether or not a term appears in the document
  2. **Term Occurrence**: Number of occurrences of a term in the document
  3. **Terms Frequency**: Attributes represent the frequency in which a term appears in the document (number of occurrences / number of words in document)
  4. **TF-IDF**: see next slide

# Extract and Represent text

Document representation:

A structure for analysis

"Bag of words"

Common representation

A vector with one dimension for every unique term in space

Term frequency (tf) =

number of times a term occurs

Good for basic search, classification

Reduce dimensionality

Term space—not ALL terms

No stop words: "the", "a"

Often no pronouns

Stemming

"Phone" = "Phones"

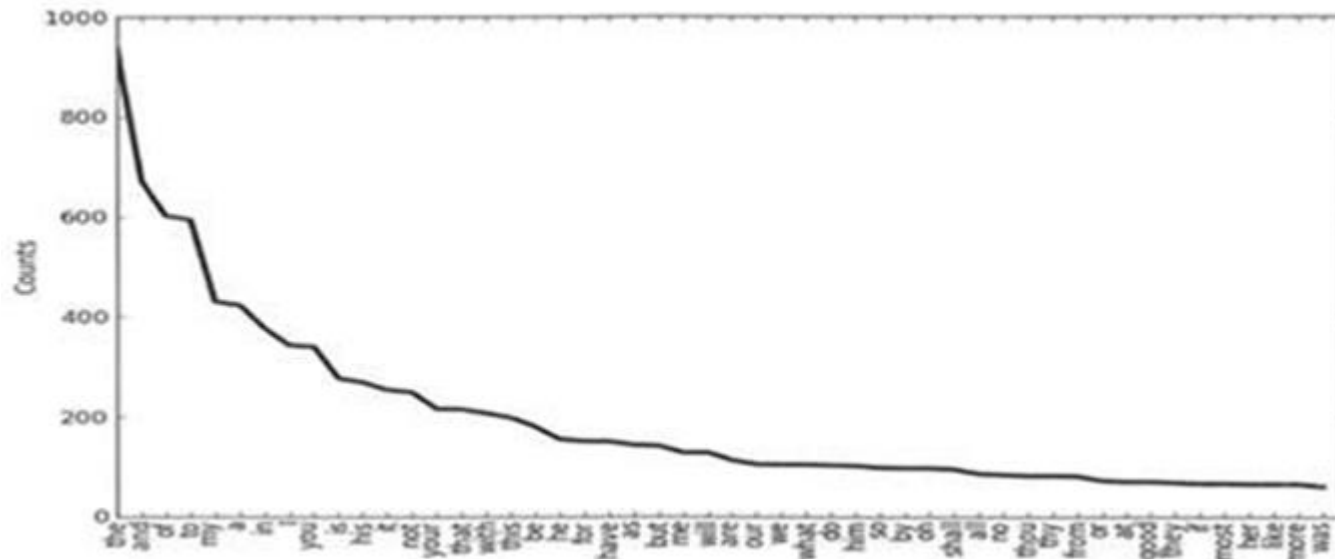
"We know what we are, but not what we may be."

Convert this sentence to a vector in the term space:

|       |   |
|-------|---|
| We    | 3 |
| what  | 2 |
| how   | 0 |
| are   | 1 |
| now   | 0 |
| may   | 1 |
| doubt | 0 |
| thing | 0 |

# Extract and Represent text

- Using single words as identifiers with the bag-of-words representation, the term frequency of each word can be calculated.
- As an example, here is a plot for terms with highest counts from Shakespeare's Hamlet.
- If you just looked at the words with the highest counts in the plot ("the," "and"), you might think these are the most important words in Hamlet. However,, these very frequent words often don't tell you much about the specific content of the play. **This example emphasizes the need for further processing techniques to extract more meaningful insights from the text.**





# Feature Generation

how often specific term  
appears in that specific  
document (frequency).

Term document matrix provides an organized matrix of observations and attributes

| Term     | Dokument |     |    |    |    |     |     |    |     |     |     |    |    |    |    |    |    |    |     |    | Σ    |
|----------|----------|-----|----|----|----|-----|-----|----|-----|-----|-----|----|----|----|----|----|----|----|-----|----|------|
|          | A        | B   | C  | D  | E  | F   | G   | H  | I   | J   | K   | L  | M  | N  | O  | P  | Q  | R  | S   | T  |      |
| oil      | 5        | 12  | 2  | 1  | 1  | 7   | 3   | 3  | 5   | 9   | 5   | 4  | 5  | 4  | 3  | 4  | 5  | 3  | 3   | 1  | 85   |
| price    | 5        | 6   | 2  | 2  | 0  | 8   | 1   | 2  | 2   | 10  | 5   | 1  | 5  | 2  | 0  | 3  | 3  | 3  | 3   | 0  | 63   |
| opec     | 0        | 15  | 0  | 0  | 0  | 8   | 1   | 2  | 2   | 6   | 5   | 2  | 2  | 4  | 0  | 0  | 0  | 0  | 0   | 0  | 47   |
| mln      | 0        | 4   | 0  | 0  | 2  | 4   | 1   | 0  | 0   | 3   | 9   | 0  | 0  | 0  | 0  | 3  | 3  | 0  | 0   | 2  | 31   |
| market   | 2        | 5   | 0  | 0  | 0  | 3   | 0   | 2  | 0   | 10  | 1   | 2  | 2  | 0  | 0  | 0  | 0  | 0  | 3   | 0  | 30   |
| barrel   | 2        | 0   | 1  | 1  | 0  | 4   | 0   | 0  | 1   | 3   | 3   | 0  | 1  | 1  | 0  | 3  | 3  | 1  | 0   | 2  | 26   |
| bpd      | 0        | 4   | 0  | 0  | 0  | 7   | 0   | 0  | 0   | 2   | 8   | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0   | 0  | 23   |
| dllrs    | 2        | 0   | 1  | 2  | 2  | 2   | 1   | 0  | 0   | 4   | 2   | 0  | 0  | 0  | 0  | 1  | 1  | 5  | 0   | 0  | 23   |
| crude    | 2        | 0   | 2  | 3  | 0  | 2   | 0   | 0  | 0   | 0   | 5   | 2  | 0  | 2  | 0  | 0  | 0  | 2  | 0   | 1  | 21   |
| saudi    | 0        | 0   | 0  | 0  | 0  | 0   | 0   | 1  | 0   | 5   | 7   | 1  | 4  | 0  | 0  | 0  | 0  | 0  | 0   | 0  | 18   |
| kuwait   | 0        | 0   | 0  | 0  | 0  | 10  | 0   | 1  | 0   | 3   | 0   | 1  | 0  | 2  | 0  | 0  | 0  | 0  | 0   | 0  | 17   |
| offici   | 0        | 0   | 0  | 0  | 0  | 5   | 1   | 1  | 0   | 1   | 4   | 3  | 1  | 0  | 0  | 0  | 0  | 0  | 1   | 0  | 17   |
| meet     | 0        | 6   | 0  | 0  | 0  | 3   | 0   | 1  | 0   | 1   | 0   | 1  | 0  | 2  | 0  | 0  | 0  | 0  | 0   | 0  | 14   |
| pct      | 0        | 0   | 0  | 0  | 2  | 0   | 2   | 2  | 2   | 1   | 0   | 0  | 1  | 0  | 0  | 1  | 1  | 0  | 0   | 2  | 14   |
| product  | 1        | 6   | 0  | 0  | 0  | 1   | 0   | 0  | 0   | 0   | 4   | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 1  | 13   |
| accord   | 0        | 0   | 0  | 0  | 0  | 0   | 0   | 0  | 0   | 5   | 1   | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 4   | 0  | 12   |
| futur    | 0        | 0   | 0  | 0  | 1  | 0   | 0   | 0  | 0   | 0   | 0   | 0  | 0  | 0  | 0  | 1  | 1  | 0  | 9   | 0  | 12   |
| minist   | 0        | 0   | 0  | 0  | 0  | 3   | 0   | 0  | 1   | 3   | 1   | 2  | 1  | 1  | 0  | 0  | 0  | 0  | 0   | 0  | 12   |
| govern   | 0        | 0   | 0  | 0  | 0  | 0   | 5   | 0  | 6   | 0   | 0   | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 0  | 11   |
| month    | 0        | 1   | 0  | 0  | 0  | 2   | 2   | 0  | 1   | 0   | 5   | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 0  | 11   |
| report   | 0        | 1   | 0  | 0  | 0  | 1   | 8   | 0  | 0   | 0   | 0   | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 0  | 11   |
| sheikh   | 0        | 0   | 0  | 0  | 0  | 3   | 0   | 0  | 5   | 2   | 0   | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0   | 0  | 11   |
| industri | 0        | 2   | 0  | 0  | 0  | 1   | 1   | 1  | 1   | 0   | 0   | 0  | 0  | 0  | 0  | 1  | 2  | 0  | 1   | 0  | 10   |
| produc   | 0        | 0   | 0  | 0  | 0  | 4   | 1   | 1  | 0   | 3   | 0   | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0   | 1  | 10   |
| quota    | 0        | 2   | 0  | 0  | 0  | 4   | 0   | 0  | 1   | 1   | 1   | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0   | 0  | 10   |
| reserv   | 0        | 0   | 0  | 0  | 3  | 0   | 0   | 0  | 1   | 0   | 0   | 0  | 0  | 0  | 0  | 3  | 3  | 0  | 0   | 0  | 10   |
| world    | 0        | 1   | 0  | 0  | 0  | 1   | 3   | 0  | 1   | 1   | 0   | 0  | 1  | 1  | 0  | 0  | 0  | 0  | 1   | 0  | 10   |
| ...      |          |     |    |    |    |     |     |    |     |     |     |    |    |    |    |    |    |    |     |    |      |
| Σ        | 48       | 204 | 34 | 39 | 46 | 219 | 219 | 73 | 161 | 180 | 208 | 57 | 61 | 54 | 56 | 68 | 89 | 44 | 147 | 32 | 2039 |

# Term Frequency (TF)

---

- Term Frequency (TF) of each word can be calculated.
- Term frequency represents the weight of each term in a document,
- It is proportional to **the number of occurrences of the term in that document.**
- Along with the Bag-of-words method, you can also use this function to calculate weights.
- Assign each term in a document a weight for that term.
- The weight of a term  $t$  in a document  $d$  is a function of the number of times  $t$  is displayed in  $d$ .
  - The weight can be set to the number of occurrences of  $t$  in  $d$ :

$$tf(t, d) = \text{count}(t, d)$$

# Representing corpus

---

- A corpus is a collection of text documents.
  - Group of news articles
  - A set of emails or tweets
- Corpus metrics
  - Volume
  - Corpus-wide term frequencies
  - Inverse document frequency (IDF)
- Challenge: a corpus is often dynamic
  - Indexes and metrics must be updated continuously

# TF-IDF term weighting scheme

- The TF-IDF weight (term frequency–inverse document frequency) is used to evaluate how important a word is to a corpus of documents.

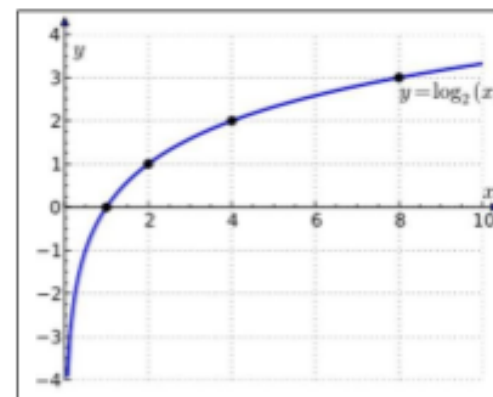
- TF: Term Frequency (see last slide)
- $N$ : total number of docs in corpus
- IDF: Inverse Document Frequency.  
 $df_i$ : the number of docs in which  $t_i$  appears

$$w_{ij} = tf_{ij} \times idf_i.$$

$$idf_i = \log \frac{N}{df_i}$$

$$TF_{t,d} = \frac{\text{Count of } t \text{ in document } d}{\text{Total number of words in } d}$$

- Gives more weight to rare words.
- Give less weight to common words (domain-specific stopwords.)



# TF-IDF term weighting scheme

## Example:

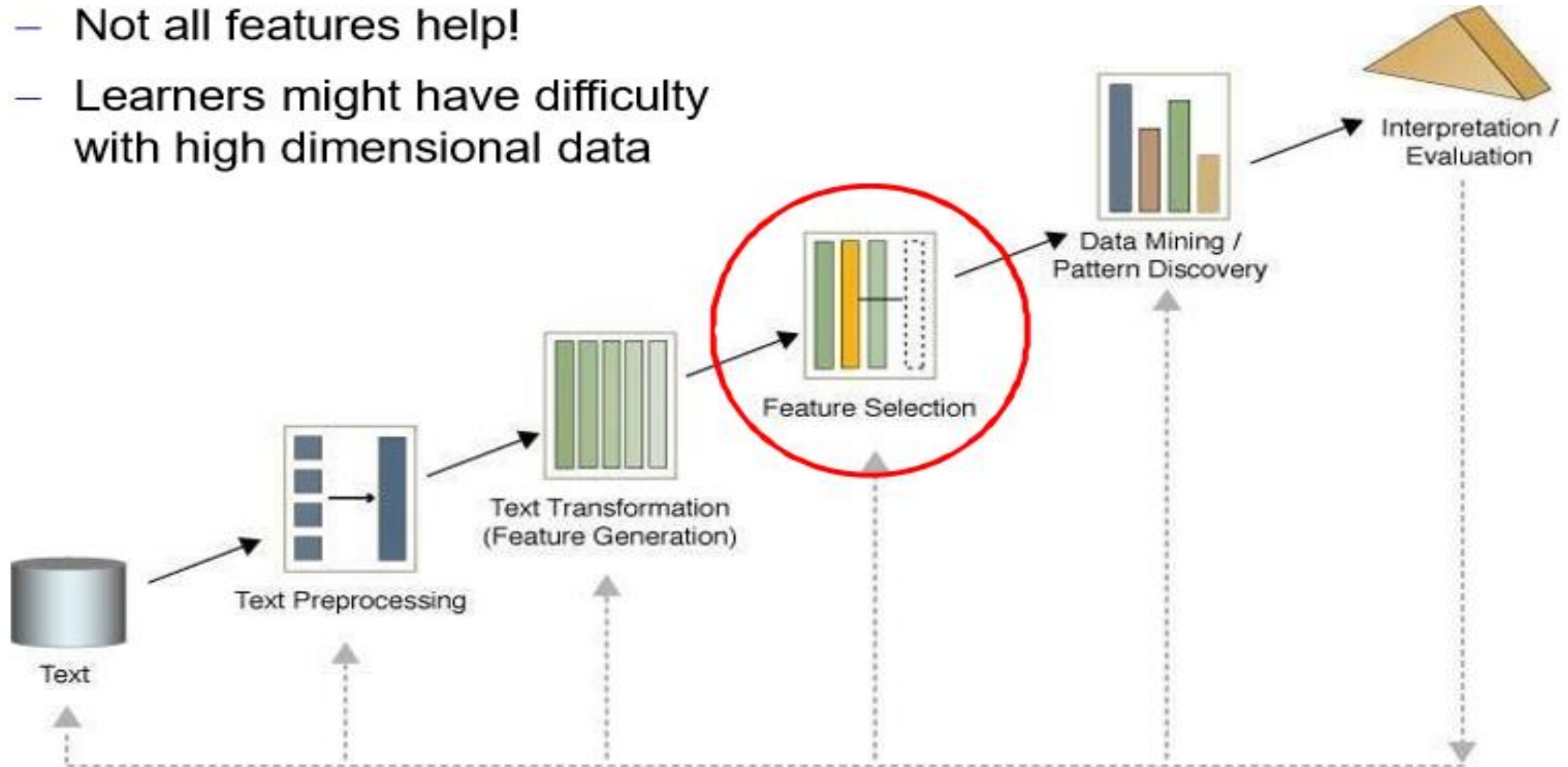
If you have a corpus of **1000 movie reviews**:

- **Word: "fantastic"**: Appears 10 times in one specific review ( $TF = 10$ ) and appears in only 50 out of the 1000 reviews ( $dfi=50$ ).
  - $idf_{\text{fantastic}} = \log(1000/50) = \log(20) \approx 1.3$
  - $W_{\text{fantastic}} = 10 \times 1.3 = 13$  (relatively high)
- **Word: "movie"**: Appears 15 times in the same review ( $TF = 15$ ) but appears in 900 out of the 1000 reviews ( $dfi=900$ ).
  - $idf_{\text{movie}} = \log(1000/900) = \log(1.11) \approx 0.05$  (very low)
  - $W_{\text{movie}} = 15 \times 0.05 = 0.75$  (relatively low)
- Even though "movie" appears more often in the review than "fantastic," "fantastic" has **a much higher TF-IDF score** because **it's a rare** word across all the reviews, making it more indicative of the sentiment or specific content of that particular review.



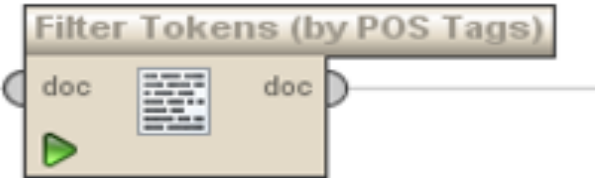
# Feature Selection

- Not all features help!
- Learners might have difficulty with high dimensional data



# Filter Tokens by POS tags

- POS tagging may be helpful for feature selection
- sometimes you want to focus on certain classes of words:

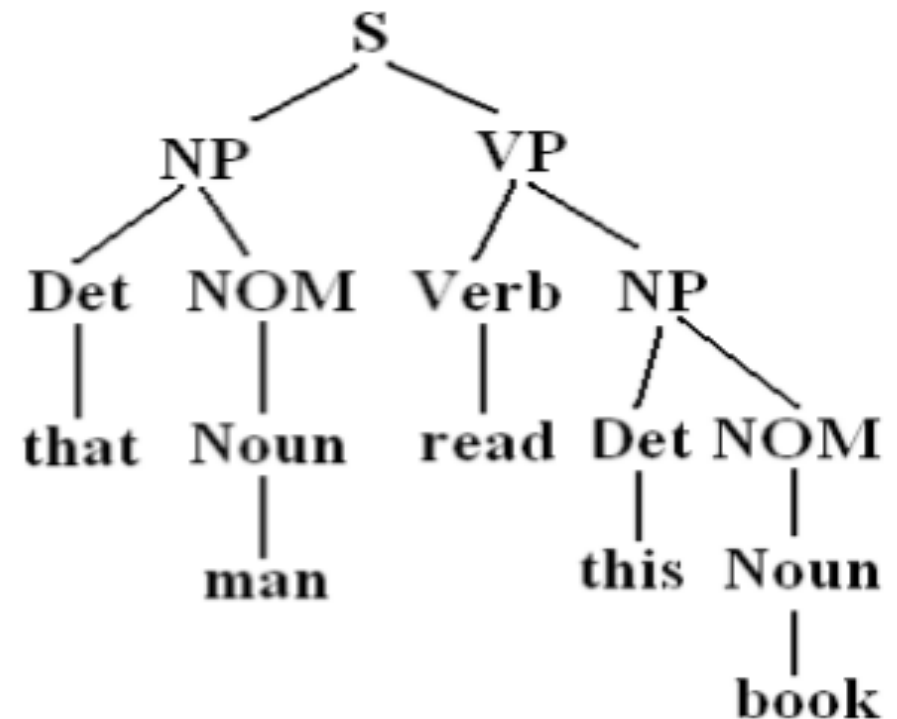


- Adjectives (JJ.) for sentiment analysis
  - good, bad, great
- Nouns (N.) for text clustering
  - red and blue cars are similar
  - red and blue trousers are similar

- Rapidminer supports

- PENN tag system for English
- STTS tag system for German
- filtering conditions are expressed as regular expressions

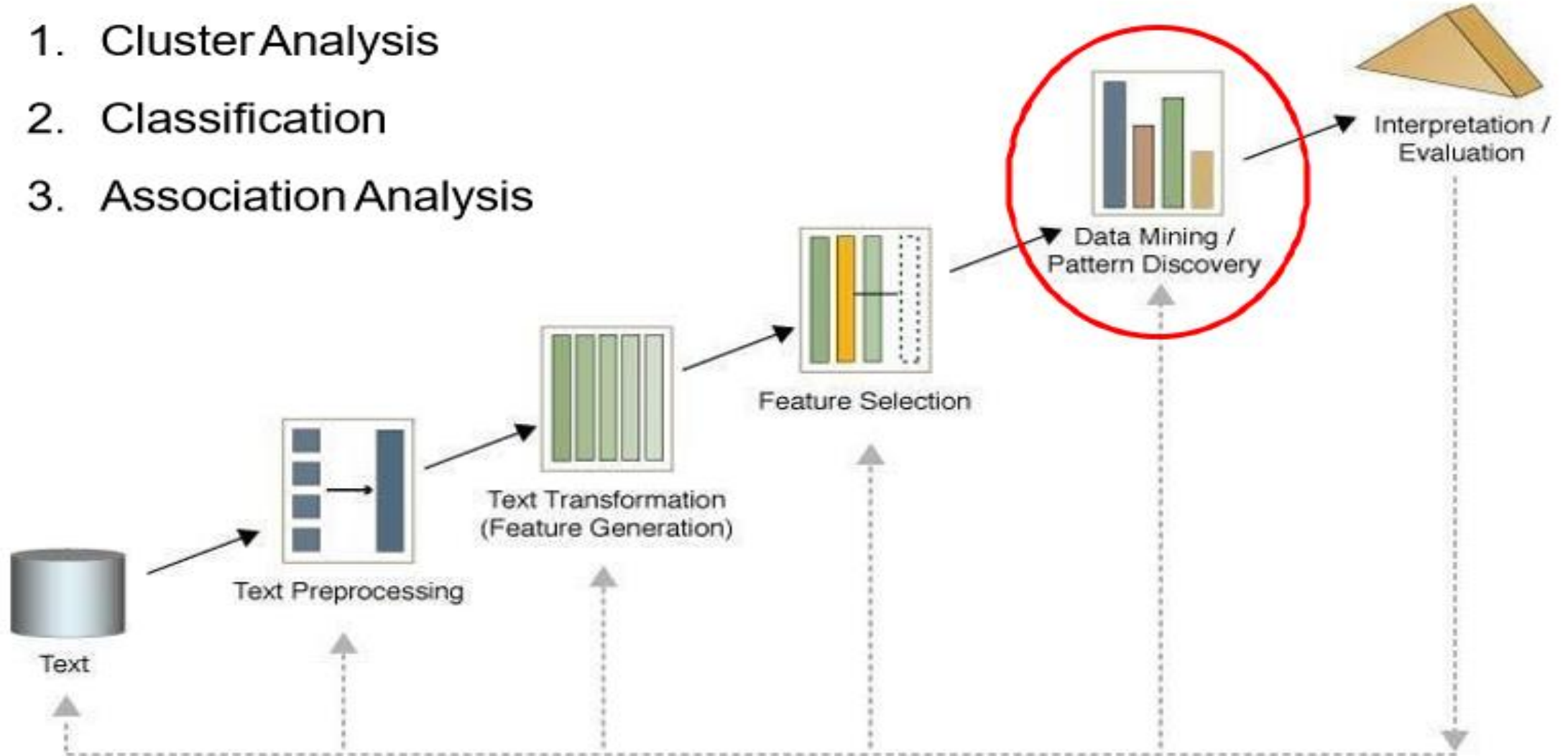
- Python: NLTK library supports PENN tag system for English



# Pattern Discovery

Methods:

1. Cluster Analysis
2. Classification
3. Association Analysis





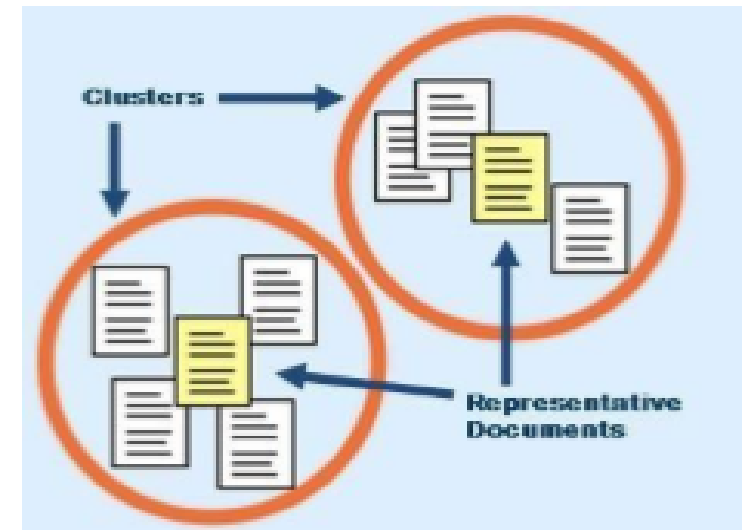
# Document Clustering

## Goal

- Given a **set of documents** and a **similarity measure** among documents find clusters such that
  - documents in one cluster are more similar to one another
  - documents in separate clusters are less similar to one another
- using some clustering algorithm

## Applications

- Topical clustering of news stories
- Email message thread identification
- Grouping of document versions



# Similarity Measure (Jaccard)

- The **Jaccard coefficient** is a popular similarity measure for vectors consisting of asymmetric binary attributes

$$dist(\mathbf{x}_i, \mathbf{x}_j) = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

Number of 11 matches / number of not-both-zero attributes values

- used together with **binary term occurrence vector**
  - 1 represents occurrence of specific word
  - 0 represents absence of specific word

# Similarity Measure (Jaccard)

- Example document set

d1 = "Saturn is the gas planet with rings".

d2 = "Jupiter is the largest gas planet".

d3 = "Saturn is the Roman god of sowing".

- Documents as binary term occurrence vectors

|    | Saturn | is | the | gas | planet | with | rings | Jupiter | largest | Roman | god | of | sowing |
|----|--------|----|-----|-----|--------|------|-------|---------|---------|-------|-----|----|--------|
| d1 | 1      | 1  | 1   | 1   | 1      | 1    | 1     | 0       | 0       | 0     | 0   | 0  | 0      |
| d2 | 0      | 1  | 1   | 1   | 1      | 0    | 0     | 1       | 1       | 0     | 0   | 0  | 0      |
| d3 | 1      | 1  | 1   | 0   | 0      | 0    | 0     | 0       | 0       | 1     | 1   | 1  | 1      |

- **d1 vector:** [1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]

- **d2 vector:** [0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0]

- Words in both d1 and d2 (M11): "is", "the", "gas", "planet" (4 words)

- Words in d1 but not d2 (M10): "Saturn", "with", "rings" (3 words)

- Words in d2 but not d1 (M01): "Jupiter", "largest" (2 words)

- Jaccard Similarity =  $M11 / (M01 + M10 + M11) = 4 / (2 + 3 + 4) = 4 / 9 \approx 0.44$

# Similarity Measure (Jaccard)

- Example document set

d1 = “Saturn is the gas planet with rings”.

d2 = “Jupiter is the largest gas planet”.

d3 = “Saturn is the Roman god of sowing”.

- Documents as binary term occurrence vectors

|    | Saturn | is | the | gas | planet | with | rings | Jupiter | largest | Roman | god | of | sowing |
|----|--------|----|-----|-----|--------|------|-------|---------|---------|-------|-----|----|--------|
| d1 | 1      | 1  | 1   | 1   | 1      | 1    | 1     | 0       | 0       | 0     | 0   | 0  | 0      |
| d2 | 0      | 1  | 1   | 1   | 1      | 0    | 0     | 1       | 1       | 0     | 0   | 0  | 0      |
| d3 | 1      | 1  | 1   | 0   | 0      | 0    | 0     | 0       | 0       | 1     | 1   | 1  | 1      |

- Jaccard similarities between the documents

- **$\text{sim}(d1, d2) = 0.44$**
- $\text{sim}(d1, d3) = 0.27$
- $\text{sim}(d2, d3) = 0.18$

# Similarity Measure (Cosine)

- Popular similarity measure for comparing weighted document vectors such as **term-frequency** or **TF-IDF vectors**

$$\cos(d_1, d_2) = \frac{d_1 \bullet d_2}{\|d_1\| \|d_2\|}$$

$$a \cdot b = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

where  $\bullet$  indicates vector dot product and  $\|d\|$  is the length of vector  $d$

$$\|a\| = \sqrt{\sum_{i=1}^N a_i^2}$$

**A higher score (closer to 1) means more similarity.**

– Example

$$d_1 = 3 \ 2 \ 0 \ 5 \ 0 \ 0 \ 0 \ 2 \ 0 \ 0$$

$$d_2 = 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 2$$

$$d_1 \bullet d_2 = 3*1 + 2*0 + 0*0 + 5*0 + 0*0 + 0*0 + 0*0 + 2*1 + 0*0 + 0*2 = 5$$

$$\|d_1\| = (3*3 + 2*2 + 0*0 + 5*5 + 0*0 + 0*0 + 0*0 + 2*2 + 0*0 + 0*0)^{0.5} = (42)^{0.5} = 6.481$$

$$\|d_2\| = (1*1 + 0*0 + 0*0 + 0*0 + 0*0 + 0*0 + 0*0 + 1*1 + 0*0 + 2*2)^{0.5} = (6)^{0.5} = 2.245$$

$$\cos(d_1, d_2) = 0.3150$$

# Similarity Measure (Cosine)

Suppose that our goal is to calculate the cosine similarity of the two documents given below.

- Document 1 = 'the best data science course'
- Document 2 = 'data science is popular'

After creating a word table from the documents, the documents can be represented by the following vectors:

|    | the | best | data | science | course | is | popular |
|----|-----|------|------|---------|--------|----|---------|
| D1 | 1   | 1    | 1    | 1       | 1      | 0  | 0       |
| D2 | 0   | 0    | 1    | 1       | 0      | 1  | 1       |

# Similarity Measure (Cosine)

- $D1 = [1, 1, 1, 1, 1, 0, 0]$
- $D2 = [0, 0, 1, 1, 0, 1, 1]$

Using these two vectors we can calculate cosine similarity. First, we calculate the dot product of the vectors:

$$D1 \cdot D2 = 1 \times 0 + 1 \times 0 + 1 \times 1 + 1 \times 1 + 1 \times 0 + 0 \times 1 + 0 \times 1 = 2$$

Second, we calculate the magnitude of the vectors:

$$\|D1\| = \sqrt{1^2 + 1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2} = \sqrt{5}$$

$$\|D2\| = \sqrt{0^2 + 0^2 + 1^2 + 1^2 + 0^2 + 1^2 + 1^2} = \sqrt{4}$$

Finally, cosine similarity can be calculated by dividing the dot product by the magnitude

$$\text{similarity}(D1, D2) = \frac{D1 \cdot D2}{\|D1\| \|D2\|} = \frac{2}{\sqrt{5}\sqrt{4}} = \frac{2}{\sqrt{20}} = 0.44721$$

# Similarity Measure (Cosine)

- A **commonly used combination** for text clustering
- Each document is represented by vectors of TF-IDF weights
- **A clustering algorithm would try to group documents with higher cosine**
- Sample document set:

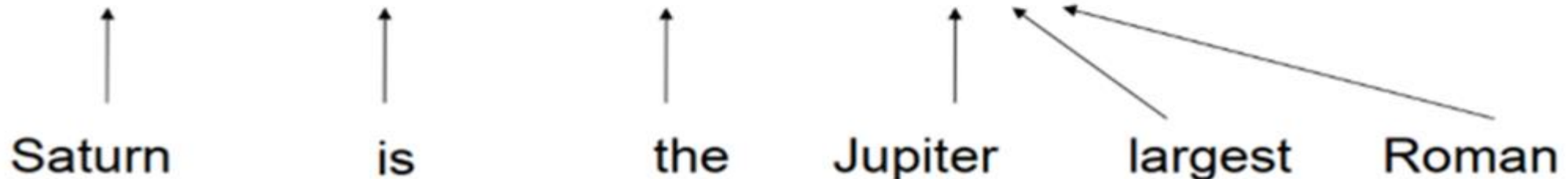
- “Saturn is the gas planet with rings”.
- “Jupiter is the largest gas planet”.
- “Saturn is the Roman god of sowing”.

$$w_{ij} = tf_{ij} \times idf_i$$

$$idf_i = \log \frac{N}{df_i}$$

- First document as TF-IDF vector:

$(1/7 * \log(3/2), 1/7 * \log(3/3), 1/7 * \log(3/3), \dots, 0, 0, 0, \dots)$





# Similarity Measure (Cosine)

- Sample document set

d1 = “Saturn is the gas planet with rings”.

d2 = “Jupiter is the largest gas planet”.

d3 = “Saturn is the Roman god of sowing”.

- Documents as TF-IDF vectors

|    | Saturn | is | the | gas  | planet | with | rings | Jupiter | largest | Roman | god  | of   | sowing |
|----|--------|----|-----|------|--------|------|-------|---------|---------|-------|------|------|--------|
| d1 | 0.03   | 0  | 0   | 0.03 | 0.03   | 0.07 | 0.07  | 0       | 0       | 0     | 0    | 0    | 0      |
| d2 | 0      | 0  | 0   | 0.03 | 0.03   | 0    | 0     | 0.08    | 0.08    | 0     | 0    | 0    | 0      |
| d3 | 0.03   | 0  | 0   | 0    | 0      | 0    | 0     | 0       | 0       | 0.07  | 0.07 | 0.07 | 0.07   |

- Cosine similarities between the documents

- $\cos(d1, d2) = 0.13$

- $\cos(d1, d3) = 0.05$

- $\cos(d2, d3) = 0.00$

$$\cos(d_1, d_2) = \frac{d_1 \bullet d_2}{\|d_1\| \|d_2\|}$$

# Document Classification

---

- Given: A collection of labeled documents (**training set**)
- Find: A model for the class as a function of the values of the features
- Goal: Previously unseen documents should be assigned a class as accurately as possible
- **Applications**
  - topical classification of news stories or web pages
  - SPAM detection
  - sentiment analysis
- **Classification methods** commonly used for text
  1. naive bayes
  2. support vector machines (SVMs)
  3. recurrent neural networks (RNNs), e.g. long short-term memory (LSTMs)
  4. but KNN or random forests may also work

# Performing sentiment analysis

---

- Sentiment analysis is the process of examining text to determine if it corresponds to negative, neutral, or positive feedback.
- To perform sentiment analysis, you can take advantage of several existing packages, which are readily available on the web.

# Polarity: Simple Sentiment Scoring

---

- **Polarity**, the measure of positive or negative intent in a writer's tone, can be calculated by sophisticated or fairly straightforward methods.
- The resulting polarity calculation is a number that is negative to represent a negative, zero to represent neutral and positive to represent positive tone.
- **Subjectivity Lexicons**
  - is a list of words associated with a particular emotional state.
  - For example, the words bad, awful and terrible can all reasonably be associated with a **negative state**.
  - In contrast, perfect and ideal can be connected with a **positive state**.

# Sentiment Analysis

- The goal of sentiment analysis is to determine the polarity of a given text at the document, sentence, or feature/aspect level
- Polarity values
  - positive, neutral, negative
  - likert scale 1 to 10
- Application examples
  - Document level
    - analysis of tweets about politicians
  - Feature/aspect level
    - analysis of product reviews



# Sentiment Analysis



- Given: A text
- Goal: Assign a class of sentiment to the text
  - e.g., positive, neutral, negative
  - e.g., sad, happy, angry, surprised
- Can be implemented as supervised classification task
  - requires training data
  - i.e., pairs like <text; sentiment>
    - <"This is a great day!", positive>
    - <"I feel really down today.", negative>
    - <"The weather is cloudy.", neutral>
    - <"I can't believe they did that!", angry>
    - <"Wow, that was unexpected!", surprised>



# Sentiment Analysis

- Labeling data for sentiment analysis
  - is expensive, like every data labeling task
- Reviews from the Web may be used as labeled data

173 of 213 people found the following review helpful

## ★★★★★ Listen Closer

Trent Reznor should just release an album with a new title, new artwork, and new song titles. But instead of actual new material, it should all just be the songs from The Downward Spiral.

It can be called There You Go, \*\*\*\*heads.

After all, it's what everyone wants.

I remember the day I bought The Downward Spiral. My first thought after...

[Read the full review >](#)

Published 1 month ago by Philip Atherton

Vs.

19 of 21 people found the following review helpful

## ★★★★☆ Good, But Not Their Best

Its funny how immediately after an established band that's been around for a while comes out with a new album all the fan-boys give reviews saying it's the greatest thing ever. I am a Nine Inch Nails fan too and have all their albums, so I'd thought I'd give my review which I hope is a little more fair.

It's an electronic based album with some guitar, bass,...

[Read the full review >](#)

Published 1 month ago by JKat

- There exist various large corpora of reviews for public download
  - Amazon Product Data by Julian McAuley: 142 million reviews from Amazon
  - WebDataCommons: 70 million reviews from 50,000 websites that use RDFa or Microdata markup

# More possibilities with text analysis

---

- Topic tagging—segment the text documents into the specific categories
  - Segment books into the right categories. There are 300,000 titles published in 2013 in the US only.
  - In the current example, the plays can be categorized into comedies, tragedies, and histories.
- Sentiment analysis refers to tasks that use statistics and natural language processing to mine opinions to identify and extract subjective information from texts.
  - This analysis is performed on Twitter to determine overall opinion on a particular trending topic.
  - Companies and brands often utilize sentiment analysis to monitor brand reputation across social media platforms.



# More possibilities with Text Analysis

---

- Topic models are statistical models that examine words from a set of documents, determine the themes over the text, and discover how the themes are associated or change over time.
- The process of topic modeling can be simplified to the following:
  1. Uncover the hidden topical patterns within a corpus.
  2. Annotate documents according to these topics.
  3. Use annotations to organize, search, and summarize texts.

# References

---

- *"Business intelligence: data mining and optimization for decision making". Vercellis, Carlo, John Wiley & Sons.*
- *"Predictive Analytics: The Power to Predict Who Will Click, Buy, Lie, or Die" by Eric Siegel*